## The Hotwire Way

Hotwire[31] is the generic name for the client-side toolkit written by the Basecamp team to power the Hey email application.[32] The goal of Hotwire is to support an application where most of the dynamic nature happens by making normal HTTP requests to the server, receiving HTML for a part of the page, and inserting that HTML in the correct place in the DOM for a page update. (The name *Hotwire* is derived from the phrase, “HTML over the wire.”)

The idea is that by moving all the logic to the server, you can replace a lot of complicated and specific client-side code with a small set of generic client-side actions that handle retrieving and managing HTML from the server. You still have to write the logic server side, but the bet here is that, at least some of the time, by making the server the source of truth, you are preventing duplicate logic on the client and the server. In particular, Turbo allows you to reuse and repurpose view code that you have already written for greater interactivity. The intention is that writing the complex logic in Ruby and Rails will be easier than writing it in the JavaScript ecosystem, and will allow you to take advantage of the Rails code that you are already writing to support more dynamic interactions. Your mileage may vary on that last part, but you are reading a book about Rails development, so it is my hope that it does become easier for you.

This is the big-picture idea:

- The server communicates with the client by sending rendered HTML, not raw data. This HTML response may contain some metadata about where to put it when it arrives.
- Any business logic the application might need should be on the server, not on the client.
- Specific client logic should be limited only to interface items that the server won’t care about.
- Where possible, client logic can be handled by calling the server for HTML